

GCD algorithms in cpp

1. find GDC using cpp. Show multiple algorithms.2



1. find GCD using cpp. Show multiple algorithms

To find the **GCD (Greatest Common Divisor)** of two integers using C++, there are **multiple algorithms** you can implement. Below, I'll show you **three common methods**:

✓ 1. Euclidean Algorithm (Iterative)

The most efficient and commonly used method.

```
cpp

#include <iostream>
using namespace std;

int gcd_iterative(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int main() {
    int a = 48, b = 18;
    cout << "GCD (Iterative): " << gcd_iterative(a, b) << endl;
    return 0;
}
```

✓ 2. Euclidean Algorithm (Recursive)

Simpler to write using recursion.

```
cpp

#include <iostream>
using namespace std;

int gcd_recursive(int a, int b) {
    if (b == 0)
        return a;
    return gcd_recursive(b, a % b);
}

int main() {
    int a = 48, b = 18;
    cout << "GCD (Recursive): " << gcd_recursive(a, b) << endl;
}
```

```
    return 0;
}
```

✓ 3. Brute Force Method

Checks all possible divisors from 1 to the minimum of the two numbers.

cpp

```
#include <iostream>
using namespace std;

int gcd_brute_force(int a, int b) {
    int gcd = 1;
    int minNum = min(a, b);
    for (int i = 1; i <= minNum; ++i) {
        if (a % i == 0 && b % i == 0)
            gcd = i;
    }
    return gcd;
}

int main() {
    int a = 48, b = 18;
    cout << "GCD (Brute Force): " << gcd_brute_force(a, b) << endl;
    return 0;
}
```

🧠 Optional: Using C++ STL (__gcd)

C++ provides a built-in function in the <algorithm> header.

cpp

```
#include <iostream>
#include <algorithm> // for __gcd
using namespace std;

int main() {
    int a = 48, b = 18;
    cout << "GCD (STL __gcd): " << __gcd(a, b) << endl;
    return 0;
}
```

 **Note:** __gcd is a GCC-specific extension. If you're writing portable C++ code, stick to your own implementation.

✓ Summary Table

Method	Time Complexity	Notes
Euclidean (Iterative)	$O(\log(\min(a, b)))$	Fast and efficient
Euclidean (Recursive)	$O(\log(\min(a, b)))$	Elegant, but uses stack space
Brute Force	$O(\min(a, b))$	Simple, but slow for large numbers
STL <code>__gcd</code>	$O(\log(\min(a, b)))$	Best for quick use with GCC

Let me know if you'd like versions for arrays or more optimized methods (e.g., binary GCD / Stein's algorithm).